ITPFE 1 — WEB SYSTEMS AND TECHNOLOGIES 2 · PRELIM PROJECT

Prelim Project: Build an App with the View Framework

Davao Oriental State University · FaCET · BSIT 3rd Year · A.Y. 2026–2027, 1st Semester

Overview

The prelim project replaces the Week 6 written exam. In groups of 3–4, you will build a working single-page app on the View framework from Week 5, then document it with a 5–8 minute video and a written report. You have one week from release to submission.

The project checks whether you can actually use what Weeks 1–5 covered, not just recall it. By the end, you should be able to:

1. Split a vanilla JavaScript app into a Model, Views and a Controller with clean boundaries.
2. Extend the base `View` class to build your own reusable view components.
3. Use modern JavaScript: ES modules, classes, array methods, destructuring and template literals.
4. Collaborate on one codebase through Git and GitHub.
5. Explain, in your own words, the code you personally wrote.

Teams and topics

Groups have 3–4 members and choose their own app idea. There is no pitch or approval step, so each group checks its own idea against the rules below before writing any code.

Your app idea must:

- Have at least one value that recomputes when the user types or clicks (like the cost splitter's per-person share)
- Need at least one view per member, so every member owns visible code
- Run without a backend (data lives in the model and `localStorage`)
- Not copy an in-class demo (cost splitter, forkify, BoardingEase)

Keep the scope small enough to finish in one week: three features that work beat six that half-work. Ideas that fit: a GWA calculator, a sari-sari store tally, a jeepney fare splitter, a study planner, a recipe scaler, a barkada trip budget.

Technical requirements

Every project uses the View framework from Week 5 in plain JavaScript: no React, Vue, jQuery or other JS libraries. CSS frameworks are allowed. The MVC boundaries below are graded strictly, because they are the point of the exam.

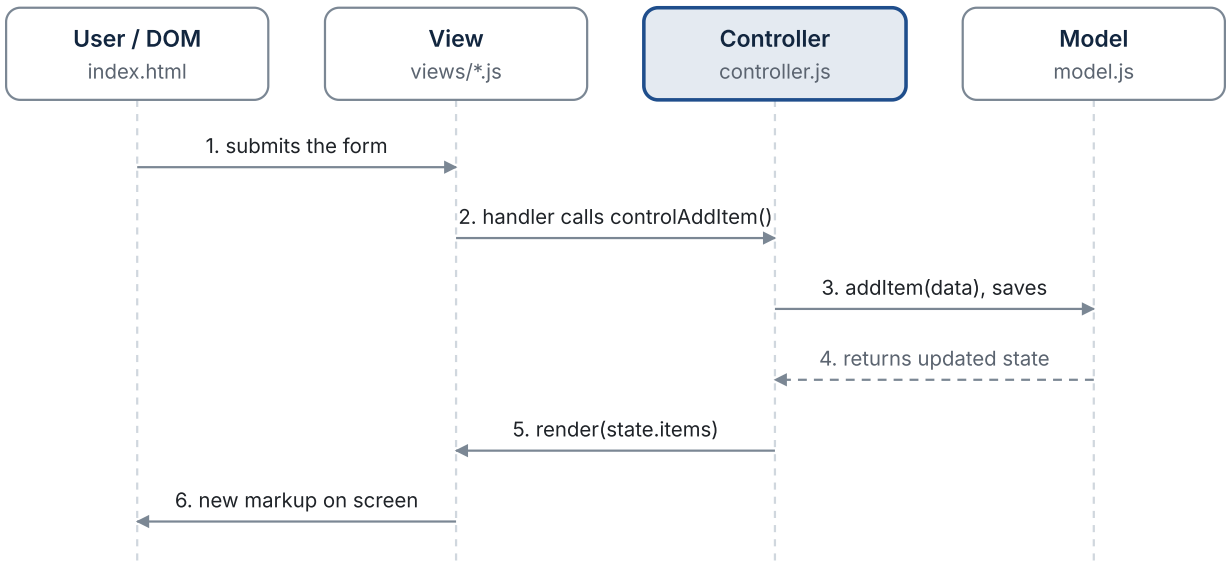
Folder structure (minimum)

```
project/
├── index.html
├── css/
│   └── style.css
└── js/
    ├── model.js
    ├── controller.js
    └── views/
        ├── View.js
        └── (one or more views per member)
```

MVC rules

Layer	Must	Must never
Model (`model.js`)	Hold all data in one exported `state` object; expose functions that change it; save to and load from `localStorage`	Touch the DOM or import any view
Views (`views/*.js`)	Extend the base `View` class; build markup from the data they are given; expose handler methods the controller subscribes to	Change `state` directly or import the model
Controller (`controller.js`)	Import the model and the views; wire each user action to a model change and a re-render	Build HTML markup itself

The controller is the only layer that talks to both the view and the model



Views never import the model and the model never touches the DOM; every change passes through a controller function. Section 5 of the written report asks you to trace exactly this path in your own app.

Minimum features

- At least one view per member, each extending `View`
- Add, edit and delete items in at least one list
- At least one form with validation that renders an error message through a view

- At least one derived value that recomputes on user input (a total, a share, a score)
- Data survives a page reload (`localStorage`)
- ES modules (`import` / `export`) throughout; no inline `onclick` in the HTML

Git and hosting

- One shared GitHub repository per group, with every member added as a collaborator
- At least 5 meaningful commits per member, written under their own GitHub account
- The app is published on GitHub Pages and the live link works on submission day

Timeline

The project runs across the four Week 6 class sessions and is due 11:59 PM on Day 7. Two checkpoints catch stalled groups early, while there is still time to recover.

When	What happens	Checkpoint
Session 1 · Lecture (1 hr)	Brief released, groups formed, app idea and view owners decided	Group members and app name submitted by end of day
Session 2 · Lab (1.5 hrs)	Repo set up; `state`, model functions and base views started	Repo link shared; every member has at least one commit
Session 3 · Lecture (1 hr)	Progress check: each group demos one feature end to end	One working View → Controller → Model → View loop
Session 4 · Lab (1.5 hrs)	Finish features, deploy to GitHub Pages, record the video	Live link works
Day 7, 11:59 PM	Final submission	All deliverables below

A group that misses the Session 2 checkpoint meets the instructor during Session 3 to rescope.

Deliverables and submission

One submission per group, sent by the group leader before the deadline. A late submission loses 10 points per day, up to 3 days.

- ☐ GitHub repository link (public, or shared with the instructor)
- ☐ Live GitHub Pages link
- ☐ Video link (YouTube unlisted or Google Drive, set to "anyone with the link")
- ☐ Written documentation as a PDF, named `GroupName_PrelimProject.pdf`

Video documentation

The video runs 5–8 minutes and every member speaks. Its main job is the code walkthrough: each member explains the code they wrote, which is how individual understanding is graded.

Part	Time	What to show
1. Introduction	~0:30	Group name, members, the app, and the problem it solves

2. Live demo	~1:30	One full user flow, including one validation error and a page reload that keeps the data
3. Trace one action	~1:00	Pick one click or input and follow it in the code: View handler → controller function → model update → view re-render
4. Member walkthroughs	~1:00 each	Each member opens their own view and controller function and explains what it does and why
5. Wrap-up	~0:30	Hardest problem the group solved, and one thing you would improve

Recording rules

- Screen recording at 1080p, with the editor font large enough to read (16 px or more)
- Each member's voice is heard during their own part; camera is optional
- No background music over speech; cuts between parts are fine
- Over 8 minutes loses 5 points; under 5 minutes usually means a part is missing

AI use policy

You may use AI tools as a tutor, but every line you submit must be code you can explain. Using AI is not what lowers your grade; submitting code you don't understand is.

Allowed:

- Asking AI to explain an error message, a concept or a piece of code
- Asking AI to review your code and suggest improvements, which you then write yourself
- Asking AI for examples of a pattern, then adapting them to your own `View` class and `state`

Not allowed:

- Submitting AI-generated code you cannot explain line by line
- Using AI without disclosing it in Section 11 of the written report

> **How it is checked:** each member explains their own code in the video walkthrough. If the walkthrough leaves doubt, the instructor may call that member in for a short follow-up consultation to explain it. Code you list as yours but cannot explain does not count toward your individual score. Undisclosed AI-generated code you cannot explain scores 0 on the individual criterion.

Grading rubric

The project is scored out of 100: 70 points shared by the group and 30 points earned by each member individually. Individual understanding carries almost a third of the grade, so a working app alone does not guarantee a high score.

Criterion	Pts	Excellent (100%)	Proficient (75%)	Developing (50%)	Beginning (25%)
MVC architecture	20	Every rule in the MVC table is followed; all views extend `View`; controller only wires	One or two boundary slips (e.g. a view reads `state`)	Several slips; model or views touch each other's job	Logic mixed in one file; MVC in name only
Functionality	15	All minimum features work, including validation, recompute and reload	One minimum feature missing or buggy	Two or three features missing or buggy	App barely runs
Code quality and Git	10	Clear names, no dead code; every member has 5+ meaningful commits	Minor clutter; one member under 5 commits	Messy code or most work in a few large commits	Code dumped in one commit
Written documentation	10	Every template section complete and accurate; trace matches the code; AI use disclosed	One or two sections thin	Several sections missing or generic	Template barely filled
Video documentation	15	All five parts within 5–8 min; trace is clear and correct	One part weak or rushed	Parts missing or hard to follow	Demo only, no code explained
Group subtotal	**70**				
Individual: code ownership	30	Explains what their own code does and why it is written that way; commits and AI disclosure match the contribution table	Explains what their code does, with small gaps on why	Reads code aloud without explaining it	Cannot explain own code, or no commits
Total	**100**				

A member's final score is the group subtotal plus their own individual score. Code a member lists as theirs but cannot explain does not count toward their individual score. A member with no commits and no walkthrough in the video gets 0 on the individual criterion and is called in for a consultation.

ITPFE 1 — WEB SYSTEMS AND TECHNOLOGIES 2 · PRELIM PROJECT

Written Documentation Template

Copy everything below into your own document, replace every [bracketed] prompt, and export it as a PDF. Keep the headings and their order so every report can be graded side by side.

Cover page

Project title	[App name]
Group name	[Group name]
Members	[Full name — role] for each member; mark the group leader
Course and section	ITPFE 1 — Web Systems and Technologies 2, [Section]
Instructor	[Instructor name]
Date submitted	[Date]
Repository	[GitHub link]
Live app	[GitHub Pages link]
Video	[Video link]

1. Project overview

- **Problem:** [Two or three sentences on the problem and who has it]
- **Users:** [Who uses the app and when]
- **Solution:** [What the app does, in one paragraph]

2. Features

Feature	What the user can do	Owner
[Feature 1]	[Description]	[Member]
[Feature 2]	[Description]	[Member]
[Feature 3]	[Description]	[Member]

3. Architecture

3.1 Folder structure. [Paste your project's folder tree]

3.2 State object. [Paste the `state` object from `model.js` with sample data, and explain each property in one line]

3.3 Responsibilities. For each layer, write two or three sentences on what it does in your app:

- **Model:** [...]
- **Views:** [...]
- **Controller:** [...]

4. Views

View class	Parent element it renders into	Handler methods it exposes	Owner
[e.g. ItemListView]	[e.g. `.item-list`]	[e.g. `addHandlerDelete`]	[Member]

5. Tracing one user action

Pick one action (e.g. adding an item) and list every step from click to screen update. Name the file and function at each step.

1. [The user ... in `…View.js`, the handler ...]
2. [The controller function ... is called with ...]
3. [The model function ... updates `state.…` and saves to `localStorage`]
4. [The controller calls `…View.render(…)`]
5. [The view rebuilds its markup and the screen shows ...]

6. Screenshots

[3–5 screenshots of the running app, each with a one-line caption. Include one showing a validation error.]

7. Testing

#	What we tested	Input	Expected result	Actual result	Pass?
1	[e.g. Empty name field]	[...]	[Error message shown, item not added]	[...]	[Yes/No]
2	[e.g. Data after reload]	[...]	[...]	[...]	[...]

List at least 6 test cases, including at least 2 invalid inputs.

8. Challenges and solutions

Challenge	How we solved it	What we learned
[Bug or design problem]	[Fix]	[Lesson]

9. Individual contributions

Member	Views and functions written	Number of commits	Share of work (%)
[Name]	[e.g. `FormView.js`, `controlAddItem`]	[From GitHub Insights]	[...]

The shares must add up to 100%, and every member signs below to confirm the table is accurate.

_____

_____

_____

_____

10. Individual reflections

Each member writes 100–150 words: what you built, what MVC made easier or harder, and what you would do differently.

11. References and AI use

List the docs, tutorials and course materials you used. Then fill in one row for every AI tool you used, or write "No AI tools used."

[Docs, tutorials and course materials used]

Member	AI tool	What you asked it	Files or functions it affected
[Name]	[e.g. ChatGPT]	[e.g. Explain why my render() shows nothing]	[e.g. `ItemListView.js`, `_generateMarkup`]

Each member must be able to explain every line listed against their name, including lines written with AI help.